

TPSM Group Assignment

Viva Preparation Guide

A K P C N Alawatta | IT23603004

Group: Data4ce | Module: IT3011

Topic: Doctor-Patient Communication Improves Patient Understanding

Complete Line-by-Line Explanation of Your R Code

How to use this guide: Read the explanation for each code block. Then read the viva Q&A at the end of each section. This guide covers every library, every function, and every result in your R file.

STEP 1 — Setup: Packages & Libraries

The Code:

```
install.packages(c("tidyverse", "janitor", "skimr", "naniar",  
"car", "lmtest", "caret", "MASS"))  
  
library(tidyverse)  
library(janitor)  
library(skimr)  
library(naniar)  
library(car)  
library(lmtest)  
library(caret)  
library(MASS)
```

What Each Library Does:

Library	What it does	Used for in your code
tidyverse	A collection of R packages for data science. Includes ggplot2 (graphs), dplyr (data manipulation), readr (reading files), tidyr (cleaning data)	Loading CSV, selecting columns, mutate, filter, ggplot graphs
janitor	Cleans messy column names — converts spaces and capitals to lowercase with underscores	clean_names() — makes column names like 'PCCScale' become 'pcc_scale'
skimr	Gives a detailed summary of your data — mean, sd, missing values, histogram in console	skim() in Step 4 descriptive analytics
naniar	Tools for working with missing (NA) values — visualise and analyse missing data patterns	Supporting missing value analysis
car	Companion to Applied Regression — provides diagnostic tools for regression models	vif() function to check multicollinearity
lmtest	Tests for linear regression assumptions	bptest() to check heteroskedasticity
caret	Classification And REgression Training — a unified framework for training and testing machine learning models in R	train(), predict(), postResample() for LM and KNN models in Step 6
MASS	Modern Applied Statistics with S — provides statistical functions including ordinal logistic regression	Loaded for potential ordinal regression; also causes select() conflict with dplyr

setwd() and getwd():

```
setwd("E:/Year 3/Y3 S1/TPSM - IT3011/dataset")  
getwd()
```

setwd() = Set Working Directory. This tells R where your dataset.csv file is saved on your computer.
getwd() = Get Working Directory — confirms R is looking in the right folder.

Viva Questions — Step 1:

Q: Why do you need to install packages?

A: R does not have all functions built in. Packages are extra tools made by the R community. You install them once, then load them with `library()` every time you open R.

Q: What is tidyverse?

A: Tidyverse is a collection of packages that work together for data science. It includes `ggplot2` for graphs, `dplyr` for data manipulation, and `readr` for reading files. We use it throughout the entire analysis.

Q: Why did you get an error with `select()` and how did you fix it?

A: The MASS package has its own `select()` function which overwrote `dplyr`'s `select()`. The fix was to use `dplyr::select()` — this tells R to use specifically `dplyr`'s version of `select`, not MASS's version.

STEP 2 — Load Data & Select Columns

```
health_df <- read_csv("dataset.csv", show_col_types = FALSE) %>% clean_names()  
dim(health_df)
```

Line-by-line explanation:

read_csv("dataset.csv") — reads the CSV file into R as a table (called a dataframe). This is from the readr package inside tidyverse.

show_col_types = FALSE — stops R from printing a long message about column types when loading.

%>% — this is called the PIPE operator. It means 'then do this next thing'. So: read the file, THEN clean the names.

clean_names() — from janitor package. Converts all column names to lowercase with underscores. Example: 'PCCScale' becomes 'pcc_scale'. This makes coding easier and avoids errors.

health_df — this is the name we gave our dataset. We chose 'health_df' (health dataframe). Sahil used 'df' — we used a more descriptive name.

dim(health_df) — shows dimensions: rows x columns. Our result: 6256 rows, 519 columns.

Your result: dim(health_df) = 6256 rows x 519 columns. This means 6,256 patients answered the survey, and there are 519 different questions/variables.

```
health_selected <- health_df %>%  
  dplyr::select(  
    pcc_scale, spent_enough_time, feelings_addressed,  
    understood_next_steps, confident_med_forms,  
    age, birth_sex, education, income_ranges  
  )
```

dplyr::select() — picks only the columns we need from the 519 total. We only keep 9 columns relevant to our research question.

Why these 9 columns? We selected them based on our research focus — communication quality (X), patient outcomes (Y), and control variables.

Variable	Role	What it measures
pcc_scale	Main X (predictor)	Pre-built Patient-Centred Communication scale score (0-100)
spent_enough_time	Supporting X	Did the doctor spend enough time with patient?
feelings_addressed	Supporting X	Were the patient's feelings addressed by doctor?
understood_next_steps	Y1 — Main Outcome	Did patient understand what to do next after visit?
confident_med_forms	Y2 — Second Outcome	Patient confidence filling medical forms
age	Control variable	Patient's age

birth_sex	Control variable	Patient's biological sex
education	Control variable	Patient's education level
income_ranges	Control variable (NEW)	Patient's household income — NOT used by Sahil

Viva Questions — Step 2:

Q: Why did you select only 9 columns out of 519?

A: We only need the variables relevant to our research question. Using all 519 columns would create noise and confusion in the analysis. We selected our main predictor (pcc_scale), two outcomes (understood_next_steps, confident_med_forms), two supporting predictors, and four control variables.

Q: What is pcc_scale and why is it your main X variable?

A: pcc_scale is a pre-built Patient-Centred Communication scale in the dataset. It is a composite score from 0 to 100 that measures the overall quality of doctor-patient communication. We use it as our main predictor because it directly measures communication quality — the core of our research topic.

Q: Why did you add income_ranges that Sahil did not use?

A: Income affects health literacy — people with lower income often have less access to healthcare information and may understand medical instructions differently. Adding income as a control variable makes our analysis more complete and is a meaningful contribution different from Sahil's work.

STEP 3 — Clean Missing Values & Recode Variables

Part A: Replace Missing Text Values with NA

```
health_df2 <- health_selected %>%  
  mutate(across(everything(), as.character)) %>%  
  mutate(across(everything(), ~ ifelse(  
    grepl("Missing data|Not Ascertained|Web partial|Don't know|inapplicable|error",  
    ., ignore.case = TRUE),  
    NA, .  
  )))
```

mutate() — modifies or creates columns in a dataframe.

across(everything(), as.character) — converts ALL columns to text (character) type first, so we can search for text patterns like 'Missing data' regardless of the original type.

grepl() — searches for a pattern inside text. Like Ctrl+F in a document. The | symbol means OR. So it searches for 'Missing data' OR 'Not Ascertained' OR 'Web partial' etc.

ignore.case = TRUE — makes the search case-insensitive. 'missing data' and 'Missing Data' both get caught.

ifelse(condition, NA, .) — if the condition is true (text matches), replace with NA. The dot (.) means 'keep the original value' otherwise.

NA — stands for Not Available. This is R's way of representing missing data.

Part B: Convert Data Types

```
health_df2 <- health_df2 %>%  
  mutate(  
    pcc_scale = as.numeric(pcc_scale),  
    age = as.numeric(age),  
    birth_sex = as.factor(birth_sex),  
    education = as.factor(education),  
    income_ranges = as.factor(income_ranges)  
  )
```

as.numeric() — converts text to numbers. pcc_scale and age need to be numbers for calculations.

as.factor() — converts text to a categorical variable (factor). birth_sex, education, income_ranges are categories, not numbers, so we treat them as factors.

colSums(is.na(health_df2)) — counts how many NA (missing) values are in each column.

Your missing value results: pcc_scale = 0 missing | understood_next_steps = 0 missing | confident_med_forms = 53 missing | age = 422 missing | birth_sex = 426 missing | education = 405 missing | income_ranges = 679 missing. This is normal — survey data always has some missing responses.

Part C: Recode Likert Scale Text to Numbers

```
health_df2 <- health_df2 %>%  
  mutate(  
    
```

```
understood_n = case_when(
  understood_next_steps == "Never" ~ 1,
  understood_next_steps == "Sometimes" ~ 2,
  understood_next_steps == "Usually" ~ 3,
  understood_next_steps == "Always" ~ 4,
  TRUE ~ NA_real_
)
```

case_when() — like a multiple if-else. It checks each condition in order and assigns the matching value.

Likert scale — a rating scale used in surveys where people choose from ordered options like Never/Sometimes/Usually/Always. We convert these to numbers 1/2/3/4 so R can do math on them.

TRUE ~ NA_real_ — this is the 'else' clause. If none of the above matched (e.g. blank or unrecognised text), assign NA.

understood_n — the new numeric column we created. The `_n` suffix means 'numeric version'.

Variable	Original text values	Recoded numbers
<code>understood_next_steps</code>	Never / Sometimes / Usually / Always	1 / 2 / 3 / 4
<code>confident_med_forms</code>	Not at all / A little / Somewhat / Very	1 / 2 / 3 / 4
<code>spent_enough_time</code>	Never / Sometimes / Usually / Always	1 / 2 / 3 / 4
<code>feelings_addressed</code>	Never / Sometimes / Usually / Always	1 / 2 / 3 / 4

```
data_final <- health_df2 %>%
  drop_na(pcc_scale, understood_n)
dim(data_final)
```

drop_na() — removes rows where `pcc_scale` or `understood_n` is missing. We only drop rows missing our key X and Y1 — not all missing values — to keep as many observations as possible.

Result: 6256 rows kept — because `pcc_scale` and `understood_n` had 0 missing values, no rows were dropped.

Viva Questions — Step 3:

Q: What is NA in R?

A: NA stands for Not Available. It represents missing data — when a survey respondent did not answer a question. R treats NA specially — most calculations skip NA by default, or you can tell R how to handle them.

Q: Why did you convert text to numbers for the Likert scales?

A: Statistical methods like correlation and regression need numbers to calculate. The original data has text like 'Never' or 'Always'. We convert these to 1, 2, 3, 4 because higher numbers represent better outcomes — this preserves the ordered nature of the scale.

Q: What is a factor in R?

A: A factor is a categorical variable — a variable that has a fixed set of possible values (categories), like Male/Female or different income ranges. R treats factors differently from numbers in regression — it automatically creates separate comparisons for each category.

STEP 4 — Descriptive Analytics

What is Descriptive Analytics? It describes and summarises your data using statistics and graphs. It answers: What does our data look like? What are the patterns? It does NOT make predictions or test hypotheses — that comes later.

Summary Statistics:

```
summary(data_final[, c("pcc_scale", "understood_n", "confident_n", "age")])  
skim(data_final[, c("pcc_scale", "understood_n", "confident_n", "age")])
```

summary() — gives Min, 1st Quartile, Median, Mean, 3rd Quartile, Max, and count of NAs for each variable.

skim() — from skimr package. More detailed version of summary — also shows standard deviation, histogram in console, and percentage of complete values.

data_final[, c(...)] — selects specific columns. The comma before c() means 'all rows, these columns'.

Your key summary results:

- pcc_scale: Min=0, Mean=76.48, Max=100 — most patients rated communication highly
- understood_n: Min=1, Mean=3.42, Max=4 — most patients usually or always understood next steps
- age: Min=18, Mean=56.43, Max=100 — wide age range in sample

Frequency Tables:

```
table(data_final$birth_sex)  
prop.table(table(data_final$birth_sex)) * 100
```

table() — counts how many times each category appears. E.g. how many Female vs Male.

prop.table() * 100 — converts counts to percentages. Result: 61.1% Female, 38.9% Male.

data_final\$birth_sex — the \$ sign accesses a specific column in a dataframe.

Graphs:

```
ggplot(data_final, aes(x = pcc_scale)) +  
  geom_histogram(bins = 20, fill = "steelblue", color = "white") +  
  labs(title = "Distribution of Patient-Centred Communication Scale",  
        x = "PCCScale Score", y = "Count")
```

ggplot() — creates a graph using the Grammar of Graphics system. You build graphs layer by layer.

aes() — stands for aesthetics. Defines what goes on which axis.

geom_histogram() — draws a histogram. bins=20 means divide the data into 20 bars.

labs() — sets the title and axis labels.

+ — in ggplot, + adds another layer to the graph.

```
ggplot(data_final, aes(x = as.factor(understood_n), y = pcc_scale,  
  fill = as.factor(understood_n))) +  
  geom_boxplot() + theme(legend.position = "none")
```

geom_boxplot() — draws boxplots. Shows median, quartiles, and outliers. We use it to compare pcc_scale scores across different levels of understanding.

as.factor(understood_n) — treats the numeric 1,2,3,4 as categories so ggplot creates separate boxes.

```
ggplot(data_final, aes(x = pcc_scale, y = understood_n)) +  
  geom_jitter(width = 0.1, height = 0.1, alpha = 0.3, color = "steelblue") +  
  geom_smooth(method = "lm", se = TRUE, color = "red")
```

geom_jitter() — like a scatter plot but adds tiny random movements so overlapping dots are visible. alpha=0.3 makes dots transparent.

geom_smooth(method='lm') — draws a regression line through the data. se=TRUE shows the confidence band around the line.

Viva Questions — Step 4:

Q: What did your histogram of pcc_scale show?

A: The histogram showed that most patients rated communication very highly — there is a large spike near 100. The distribution is left-skewed, meaning more patients gave high communication scores than low ones.

Q: What did your boxplot show?

A: The boxplot showed that as understanding of next steps increases (1 to 4), the pcc_scale scores also increase. Patients who 'Always' understood next steps had higher communication scores. This visually supports our hypothesis.

Q: What is the difference between summary() and skim()?

A: summary() gives basic statistics: min, max, mean, median, quartiles. skim() gives all of that plus standard deviation, percentage complete (non-missing), and a mini histogram. skim() is more detailed and from the skimr package.

STEP 5 — Inferential Analytics

What is Inferential Analytics? It uses statistical tests to draw conclusions from your sample about the wider population. It answers: Is there a real relationship, or could it be due to chance? It includes correlation tests and regression models.

Spearman Correlation:

```
cor.test(data_final$pcc_scale, data_final$understood_n, method = "spearman")
```

cor.test() — tests the correlation between two variables.

method = 'spearman' — Spearman correlation is used for ordinal data (ranked scales like Likert). It does not assume normal distribution. Sahil also used Spearman — it is the appropriate method for this type of data.

Why not Pearson? Pearson correlation assumes both variables are normally distributed and continuous. Our understood_n is a 1-4 Likert scale, which is ordinal, so Spearman is more appropriate.

Your result: rho = 0.841, p-value < 0.001 • rho = 0.841 means a VERY STRONG positive correlation. When communication quality is higher, understanding of next steps is also higher. • p < 0.001 means this result is statistically significant — it did NOT happen by chance. • The 'ties' warning is normal — it appears because Likert scales have many repeated values (ties).

Simple Linear Regression (model1):

```
model1 <- lm(understood_n ~ pcc_scale, data = data_final)
summary(model1)
```

lm() — Linear Model function. Fits a straight line through the data.

understood_n ~ pcc_scale — the ~ means 'is predicted by'. So: understood_n is the outcome (Y), pcc_scale is the predictor (X).

summary(model1) — shows the regression results: coefficients, p-values, R-squared.

Model1 results: pcc_scale coefficient = 0.0282, p < 0.001, R-squared = 0.759 Meaning: For every 1-point increase in pcc_scale, understood_n increases by 0.028 points. R-squared = 0.759 means pcc_scale alone explains 75.9% of the variation in understanding.

Multiple Regression (model2):

```
model2 <- lm(understood_n ~ pcc_scale + age + birth_sex + education + income_ranges,
data = data_final)
summary(model2)
```

Multiple regression — like simple regression but with multiple predictors. We add control variables (age, sex, education, income) to see if pcc_scale still matters after controlling for these.

Control variables — variables we include not because we care about them directly, but to remove their influence so we can see the 'clean' effect of pcc_scale.

Model2 key results: • pcc_scale: coefficient = 0.0279, $p < 0.001$ (***) — STILL highly significant after controls • age: $p = 0.093$ (marginal) — slight effect • birth_sex, education, income_ranges: all $p > 0.05$ — NOT significant • R-squared = 0.755 — model explains 75.5% of variance • KEY FINDING: Only pcc_scale is significant. Demographics do not explain understanding once communication is accounted for.

Regression Diagnostics:

```
vif(model2) # Multicollinearity
bptest(model2) # Heteroskedasticity
shapiro.test(sample(residuals(model2), 5000)) # Normality of residuals
par(mfrow = c(2, 2)); plot(model2); par(mfrow = c(1, 1))
```

Test	Function	What it checks	Your result	Interpretation
VIF	<code>vif()</code>	Multicollinearity — are predictors too correlated with each other?	All VIF near 1 (max ~1.5)	NO multicollinearity problem. VIF below 5 is acceptable.
Breusch-Pagan	<code>bptest()</code>	Heteroskedasticity — does variance of errors change across values?	BP = 452.83, $p < 0.001$	Heteroskedasticity present. Common in large datasets. Mention it as a limitation.
Shapiro-Wilk	<code>shapiro.test()</code>	Normality — are residuals normally distributed?	$p < 0.001$	Not perfectly normal. Expected with $n=6256$. Central Limit Theorem means regression is still valid.

`par(mfrow = c(2,2))` — arranges 4 plots in a 2x2 grid. `plot(model2)` draws 4 standard regression diagnostic plots.

`par(mfrow = c(1,1))` — resets back to single plot display.

Viva Questions — Step 5:

Q: Why did you use Spearman and not Pearson correlation?

A: Spearman correlation is designed for ordinal data — data that has an order but the gaps between values are not necessarily equal (like Never=1, Sometimes=2, Usually=3, Always=4). Pearson assumes continuous, normally distributed data. Since our outcome variable is a Likert scale, Spearman is the correct choice.

Q: What does $\rho = 0.84$ mean in your Spearman test?

A: Rho is the correlation coefficient. It ranges from -1 to +1. Our rho = 0.84 means a very strong positive relationship — when communication quality (pcc_scale) is higher, patients are more likely to understand their next steps. A value above 0.7 is generally considered a strong correlation.

Q: What does R-squared = 0.755 mean in your regression?

A: R-squared tells us how much of the variation in the outcome (understood_n) is explained by our model. 0.755 means our model explains 75.5% of the variation in patient understanding. This is a very high R-squared, showing pcc_scale is a strong predictor.

Q: What is VIF and what did your result show?

A: VIF stands for Variance Inflation Factor. It checks if predictors are too correlated with each other (multicollinearity). If VIF is above 5-10, there is a problem. Our VIF values were all close to 1, meaning no multicollinearity — all predictors provide independent information.

Q: Why is income_ranges not significant in your regression?

A: After controlling for communication quality (pcc_scale), income does not have an additional significant effect on understanding. This actually supports the main finding — communication quality is so dominant a predictor that other factors become non-significant once it is included.

STEP 6 — Predictive Analytics (LM vs KNN)

What is Predictive Analytics? We train a model on past data (training set) and test how well it predicts new, unseen data (test set). This measures real-world usefulness of the model.

Train-Test Split:

```
model_data <- data_final %>%
  drop_na(pcc_scale, age, birth_sex, education, income_ranges, understood_n)
set.seed(42)
split_idx <- createDataPartition(model_data$understood_n, p = 0.8, list = FALSE)
train_set <- model_data[split_idx, ]
test_set <- model_data[-split_idx, ]
```

drop_na() — removes rows with ANY missing value in the listed columns. Machine learning models cannot handle missing values.

set.seed(42) — sets a random seed so results are reproducible. Every time you run this code, you get the same random split. The number 42 is arbitrary — any number works.

createDataPartition() — from caret. Splits data into train (80%) and test (20%) while keeping the same proportion of outcome values in both sets.

p = 0.8 — 80% of data goes to training, 20% to testing.

model_data[split_idx,] — selects rows in the split index (training rows). The **[-split_idx,]** selects everything NOT in the index (test rows).

Your split result: 5502 total rows after dropping NAs | train_set = 4403 rows (80%) | test_set = 1099 rows (20%)

Model A: Linear Regression:

```
lm_model <- train(
  understood_n ~ pcc_scale + age + birth_sex + education + income_ranges,
  data = train_set,
  method = "lm",
  trControl = trainControl(method = "cv", number = 5)
)
lm_predictions <- predict(lm_model, newdata = test_set)
lm_results <- postResample(lm_predictions, test_set$understood_n)
```

train() — from caret package. Trains a model. method='lm' means Linear Regression.

trainControl(method='cv', number=5) — uses 5-fold Cross Validation. This splits training data into 5 parts, trains on 4, tests on 1, repeats 5 times. Makes the model evaluation more reliable.

predict() — uses the trained model to predict understood_n for the test set.

postResample() — compares predictions to actual values and gives 3 performance metrics.

Model B: K-Nearest Neighbours (KNN):

```
knn_model <- train(
  understood_n ~ pcc_scale + age + birth_sex + education + income_ranges,
```

```
data = train_set,
method = "knn",
trControl = trainControl(method = "cv", number = 5),
tuneLength = 5
)
```

method = 'knn' — K-Nearest Neighbours. A completely different type of model to linear regression. Instead of finding a formula, KNN looks at the K most similar data points and averages their outcomes to make a prediction.

tuneLength = 5 — automatically tries 5 different values of K (number of neighbours) and picks the best one.

Why KNN instead of Sahil's Random Forest? Using a different model makes our analysis independently different. KNN is a non-parametric method — it makes no assumptions about the distribution of data.

Model Performance Metrics Explained:

Metric	What it means	LM result	KNN result	Better?
RMSE	Root Mean Square Error — average prediction error. LOWER is better.	0.337	0.334	KNN
R-squared	Proportion of variance explained. HIGHER is better. 1.0 = perfect.	0.780	0.784	KNN
MAE	Mean Absolute Error — average absolute difference between predicted and actual. LOWER is better.	0.234	0.187	KNN

Conclusion: KNN outperforms Linear Regression on all 3 metrics (lower RMSE, lower MAE, higher R-squared). Both models perform well (R-squared ~0.78), confirming that pcc_scale is a strong predictor of patient understanding.

Viva Questions — Step 6:

Q: Why do you split data into train and test sets?

A: If we train and test on the same data, the model will seem perfect because it already 'saw' the answers. By testing on separate unseen data, we get an honest measure of how well the model would perform in real life on new patients.

Q: What is K-Nearest Neighbours (KNN)?

A: KNN is a machine learning algorithm that makes predictions by finding the K most similar data points (neighbours) in the training data and averaging their outcome values. For example, if K=5, it finds the 5 patients most similar to a new patient and averages their understanding scores to predict the new patient's score. It makes no assumptions about the shape of the data.

Q: What is cross-validation and why did you use it?

A: 5-fold cross validation splits the training data into 5 equal parts. The model trains on 4 parts and tests on 1 part, repeating 5 times with a different test part each time. The average performance across 5 folds is more reliable than a single train-test split. We used it to get a more honest estimate of model performance.

Q: Which model performed better and why?

A: KNN performed slightly better than Linear Regression on all three metrics — lower RMSE (0.334 vs 0.337), higher R-squared (0.784 vs 0.780), and lower MAE (0.187 vs 0.234). This suggests that the relationship between communication quality and patient understanding may not be perfectly linear, and KNN captures some non-linear patterns better.

Overall Findings & How to Present Them

Your research question was: Does doctor-patient communication quality predict a patient's ability to understand their next steps?

Analysis	Result	What to say in viva
Spearman Correlation	$\rho = 0.84, p < 0.001$	Very strong positive correlation. Higher communication quality strongly associated with better understanding.
Simple Regression	R-squared = 0.759, $p < 0.001$	pcc_scale alone explains 75.9% of variation in patient understanding.
Multiple Regression	pcc_scale significant (***), controls not significant, R-squared = 0.755	Communication quality is the dominant predictor. Demographics (age, sex, education, income) do not add significant explanatory power once communication is included.
Predictive: LM	RMSE=0.337, R2=0.780, MAE=0.234	Linear model predicts understanding well with 78% accuracy on unseen data.
Predictive: KNN	RMSE=0.334, R2=0.784, MAE=0.187	KNN performs slightly better, suggesting some non-linear patterns in the data.

How to Answer 'What is your conclusion?'

"Our analysis strongly supports the statement that doctor-patient communication improves patient understanding. The Spearman correlation of 0.84 shows a very strong positive relationship between communication quality and understanding of next steps. The regression model confirms this — pcc_scale is the only statistically significant predictor ($p < 0.001$), explaining 75.5% of variance even after controlling for age, sex, education, and income. The predictive models further validate this — both Linear Regression and KNN achieved around 78% R-squared on unseen test data, with KNN performing slightly better. Our key finding is that communication quality matters far more than demographic factors in predicting whether patients understand what to do after a medical visit."

Tough Viva Questions & Answers:

Q: How is your analysis different from your group member Sahil's?

A: Sahil manually created a communication score from 3 items. I used the pre-built pcc_scale. Sahil's outcome was understanding test results; mine is understanding next steps and confidence with medical forms. I added income as a control variable. For inferential analytics I used Spearman and linear regression same as Sahil, but for predictive analytics I used KNN instead of Sahil's Random Forest.

Q: What are the limitations of your analysis?

A: Three main limitations: First, heteroskedasticity was detected (bptest significant), meaning the variance of errors is not constant — this can affect the reliability of p-values. Second, residuals were not perfectly normally distributed, though with $n=6256$ the Central Limit Theorem reduces this concern. Third, the data is cross-sectional — we cannot prove causation, only association. We cannot say communication causes better understanding, only that they are strongly related.

Q: Why does a high R-squared not mean causation?

A: R-squared only tells us how well our model fits the data statistically. Causation requires controlled experiments where we change one variable and measure the effect. Our data is observational — patients with better doctors may also differ in other ways we did not measure. There could be confounding variables. So we can only say there is a strong association, not that communication directly causes better understanding.

Q: What does confident_med_forms show and why is it not your main outcome?

A: confident_med_forms had 4,260 missing values out of 6,256 — meaning only about 32% of respondents answered this question. This makes it unreliable as a main outcome. understood_next_steps had zero missing values and is directly aligned with our research question about post-visit understanding, making it our primary outcome.

Quick Reference — Key Terms & Numbers

Term	Simple definition
Predictor / X variable	The variable we use to predict. Here: pcc_scale (communication quality)
Outcome / Y variable	The variable we are trying to predict. Here: understood_n
Control variable	Variable included to remove its influence so we can see the clean effect of X. Here: age, sex, education, income
Likert scale	Survey rating scale with ordered options. E.g. Never/Sometimes/Usually/Always. We convert to numbers 1/2/3/4.
NA	Not Available — missing data in R
p-value	Probability that the result is due to chance. $p < 0.05$ means significant. $p < 0.001$ means very highly significant.
R-squared	Proportion of variation in Y explained by the model. $0.755 = 75.5\%$ explained.
Spearman rho	Correlation coefficient for ranked/ordinal data. Ranges -1 to +1. Our result: 0.84 (very strong positive)
RMSE	Root Mean Square Error — average prediction error. Lower is better. Our LM: 0.337, KNN: 0.334
VIF	Variance Inflation Factor — checks multicollinearity. Below 5 is fine. Our results: all near 1.
Heteroskedasticity	When error variance changes across values. Detected in our model (bptest $p < 0.001$). Noted as limitation.
Cross-validation	Method to evaluate model reliability. 5-fold CV trains 5 times on different parts of data.
KNN	K-Nearest Neighbours. Predicts by averaging K most similar data points. Our Model B in Step 6.
Train/Test split	80% of data to train model, 20% to test on unseen data. Gives honest performance measure.

Good luck in your viva! Remember: you ran this code yourself, you got these results, and you understand what each step does. Be confident — the numbers support your research question strongly.